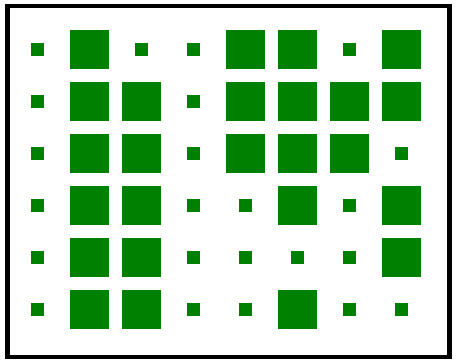
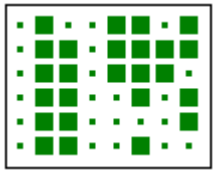


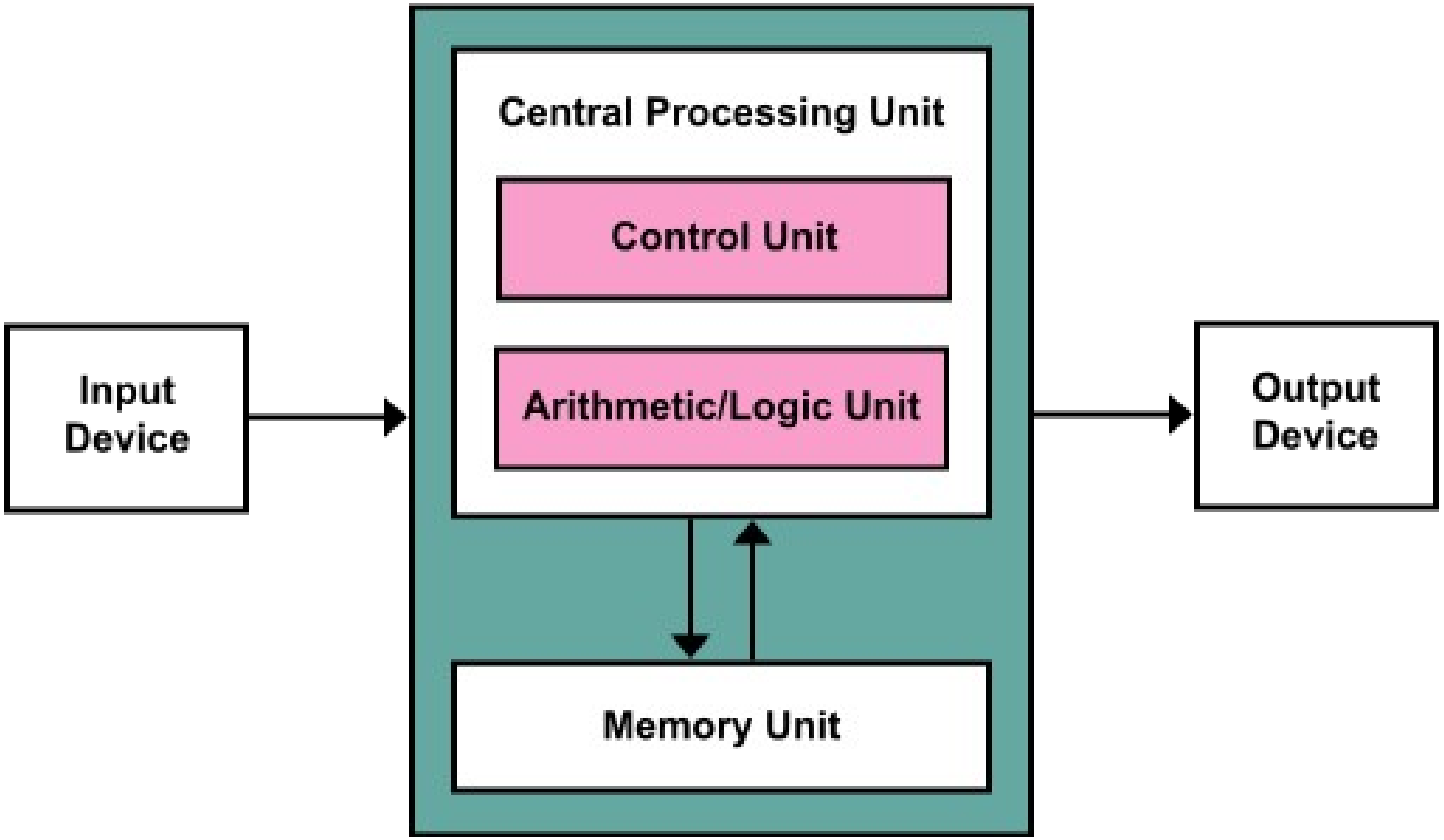
Class 4
von Neumann Architecture, Imperative Languages

CS 330
Programming Languages and Compilers

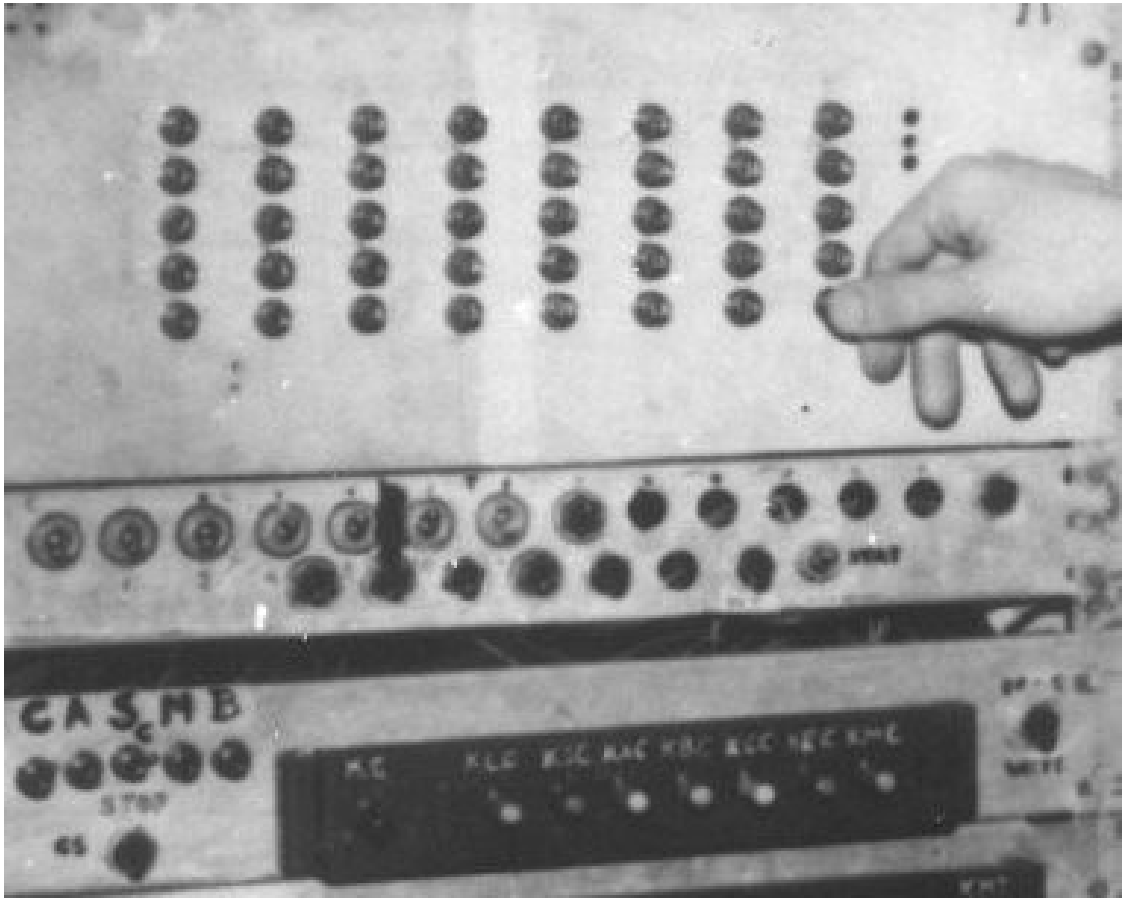
How Is A Computer Architected?



- (John) von Neumann Architecture (1945), the basis of modern computers

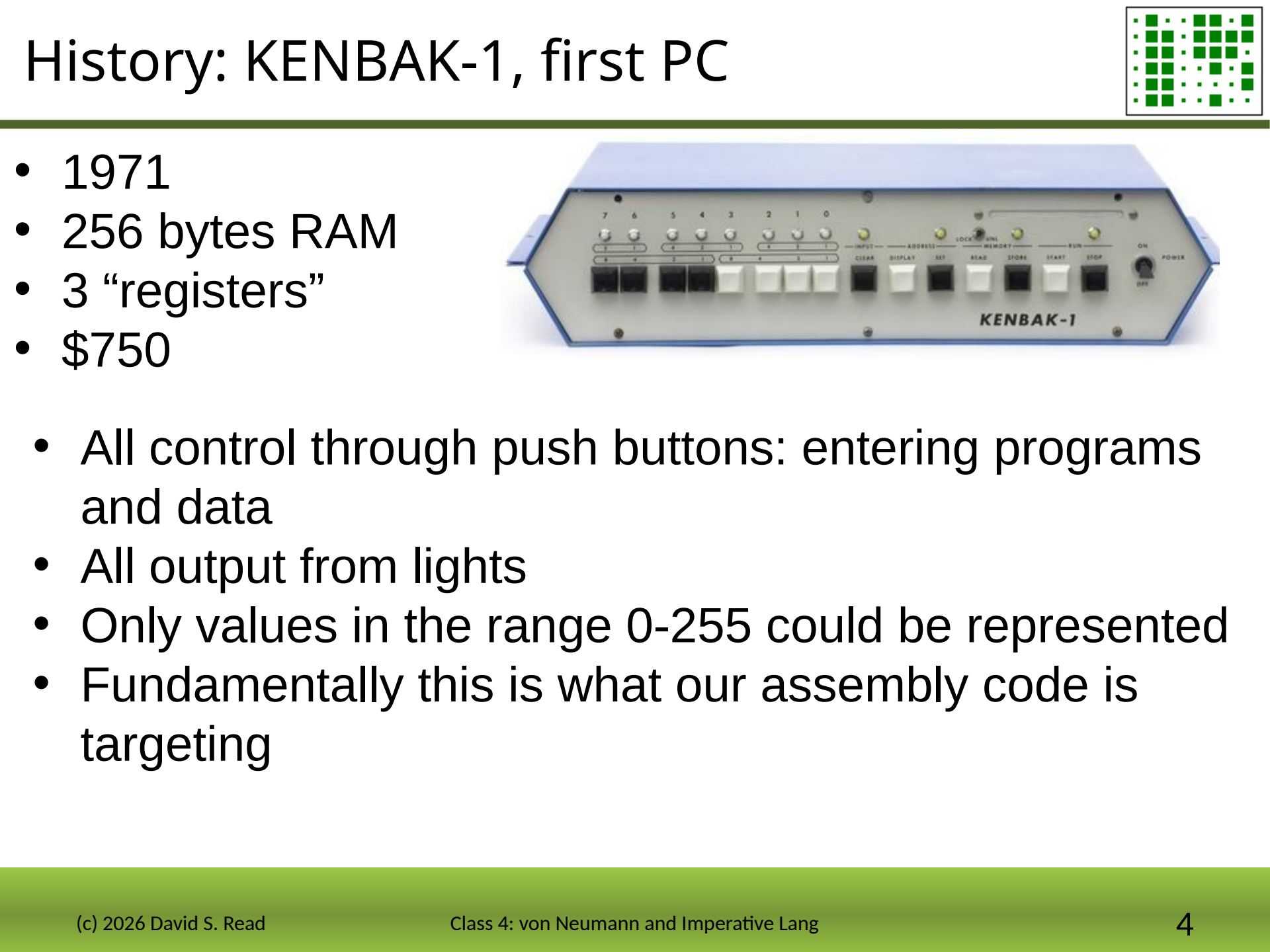


History: von Neumann Applied



- Manchester Small-scale Experimental Machine, 1948, was the first computer that held programs in software
- “Stored program computers”
- Early versions were programmed by setting switches to insert instructions into memory

History: KENBAK-1, first PC



- 1971
- 256 bytes RAM
- 3 “registers”
- $750

- All control through push buttons: entering programs and data
- All output from lights
- Only values in the range 0-255 could be represented
- Fundamentally this is what our assembly code is targeting

History: Micral, First Microprocessor PC

- 1973
- Intel 8008
- 500 kHz
- $1,750



- Originally programmed using panel switches or paper tape reader
- Designed for process control applications
- 5x cheaper than competing mini-computers
- Expanded over time to include a screen, keyboard, and floppy disks

By Photograph by Rama, Wikimedia Commons, Cc-by-sa-2.0-fr, CC BY-SA 2.0 fr, https://commons.wikimedia.org/w/index.php?curid=15530376

PCs Evolve (1977-1980s)

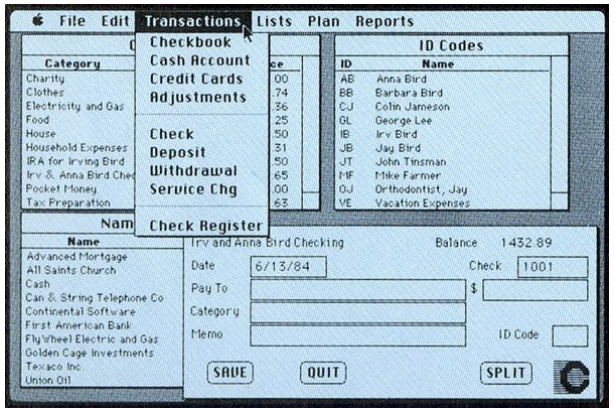










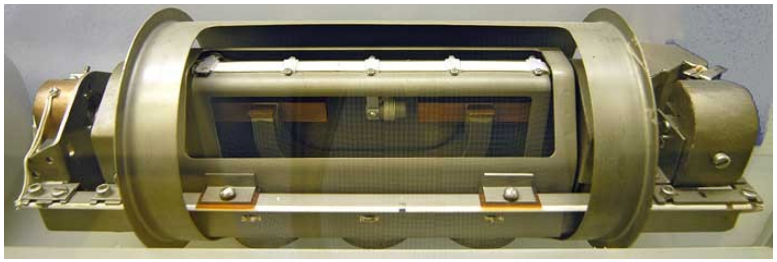


Input and Output in the 21st Century





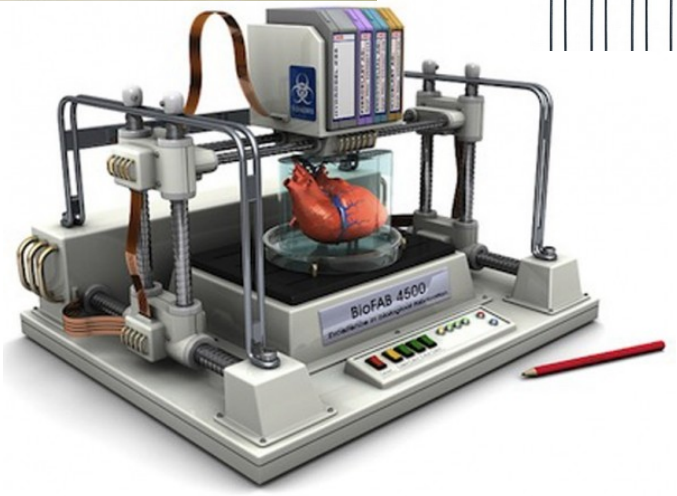








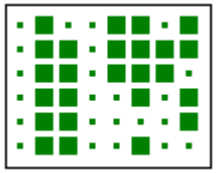






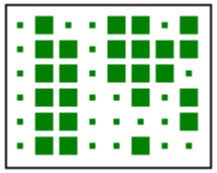


A von Neumann Computer



- Moves numeric values between CPU and RAM, carries out basic mathematical operations, and compares values
- Only instructions the computer “knows” are built into its CPU (hardware instruction set)
  - Instructions tell the CPU to do things like read values, comparing values, write values, mathematically manipulate values, and jump to other instructions
- Programming at this level is tedious and very error prone: assembly → machine

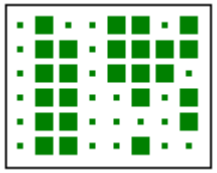
Assembly Code Example



```
.data
x: .quad 0
.text
.global main
main:
 pushq %rbp
 movq %rsp, %rbp
 pushq %rbx
 movq x, %rbx
 movq $5, %r10
 cmpq %r10, %rbx
 movq $1, %rbx
 jg .L2
 movq $0, %rbx
.L2:
# END relational operator
 movq $0, %r10
```

This is the sort of code our compiler's code generator will produce (Chapter 11)

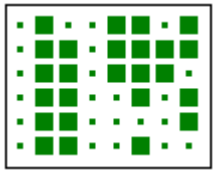
Von Neumann Bottleneck



- This architecture creates a fetch-execute cycle for program execution
  - Uses a program counter to know which instruction is next to be fetched

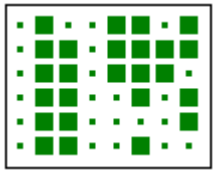
```
while (true) {
    fetch, increment, decode, execute
}
```

- Transmission between memory and CPU takes longer than executing instructions within the CPU (pipeline is slower than CPU)
  - How is that a bottleneck?



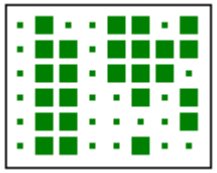
Imperative Language Concepts

All Programs Becomes CPU Steps



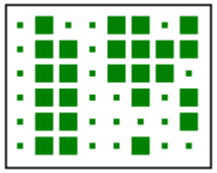
- The computer only knows how to carry out predefined operations built into the CPU
  - Programming languages are designed to make writing those steps easier *for humans*
- To be useful, our code must be converted into the primitive operations defined in the CPU, and generated in an order to run properly using the fetch-execute cycle

Aligning Language to von Neumann



- Imperative languages are designed to make the conversion process into CPU steps “easy”
  - Mapping von Neumann design concepts into language features
    - (syntax and semantics)
- For example, we control the fetch-execute order by the arrangement of statements
  - True of all imperative programming languages (like Python, C)
    - 1st generation is machine language
    - 2nd generation is assembly language
    - 3rd generation are imperative, OO, and functional

Imperative Lang Concept: Keyword

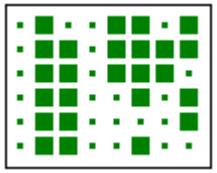


- Define the built-in operations of the language
  - Typically 30-50
- Map to one or more CPU instructions

```
if (x > 5)
    y = 10;
```

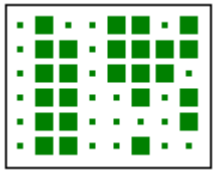
```
movq x, %rbx
movq $5, %r10
cmpq %r10, %rbx
movq $1, %rbx
jg .L2
movq $0, %rbx
.L2:
movq $0, %r10
cmpq %rbx, %r10
je .L1
movq $10, y
.L1:
```

Imperative Lang Concept: Type



- The forms of data that the language understands
    - E.g. integer, floating point number, string (text), date, complex, array
- Map to (usually more restricted) types known to CPU (often int, float, and text pointer)

Imperative Lang Concept: Identifier

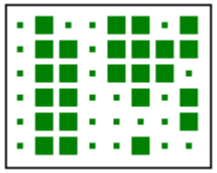


- Variables and Function Names
  - Variables are storage locations for values
    - Map to memory locations (RAM) and CPU registers
  - Function names become labels in the assembly code

```
int x = 0;
main() {
    x = 5;
}
```

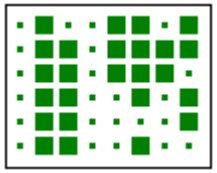
```
.data
x: .quad 0
.text
.global main
main:
pushq %rbp
movq %rsp, %rbp
movq $5, x
movq %rbp, %rsp
popq %rbp
ret
```

Imperative Lang Concept: Operator



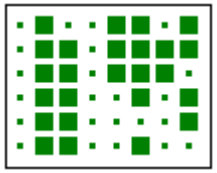
- Built in numeric, Boolean, and text operations
    - E.g. Assignment, relational and logical testing, math operations, exponentiation
- Translated to steps in the CPU for operations not directly supported
    - Often only basic mathematical and relational operations are directly implemented in the CPU's instruction set

Imperative Lang Concept: Expression



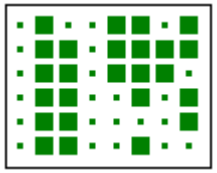
- High-level language concept
  - Set of variables, constants and operators
- No direct representation in assembly code
  - Simply leads to a list of assembly operations

Imperative Lang Concept: Statement



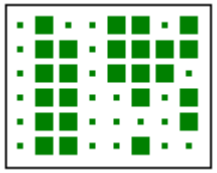
- A high-level language concept (not CPU concept)
    - We picture it as one complete unit of work (atomic unit)
- Assembly code steps may not run together
    - Becomes a concern with multi-process or multi-threaded programs

Imperative Lang Concept: Block



- A high level language concept (not CPU concept)
    - Group of statements to be executed as a unit, e.g. treated as a single statement
- As with statements, same concerns with atomicity

Imperative Lang Concept: Decision

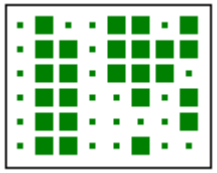


- Block that is executed if Boolean test is true
- Becomes comparisons and jumps in the assembly code

```
if (x > 5)
    y = 10;
```

```
movq x, %rbx
movq $5, %r10
cmpq %r10, %rbx
movq $1, %rbx
jg .L2
movq $0, %rbx
.L2:
movq $0, %r10
cmpq %rbx, %r10
je .L1
movq $10, y
.L1:
```

Imperative Lang Concept: Loop

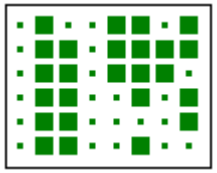


- Allow a statement to execute multiple times
  - Iteration
- Becomes assignments, comparisons, and jumps in the assembly code
  - A decision that has a jump going “backward” in the assembly code

```
int x = 0
while (x < 4)
    x = x + 1;
```

```
.data
x .quad 0
movq x, %rbx
movq $4, %r10
.L1:
cmpq %r10, %rbx
jge: .L2
addq $1, %rbx
movq %rbx, x
jmp .L1
.L2:
```

Imperative Lang Concept: Function



- A named block that can be executed from another location in the program
    - Values may be “passed in” for use in the block
    - A value may be returned when the block ends
- Becomes a labeled group of assembly code instructions
    - A call in the imperative language becomes a jump instruction in the assembly code
    - Reserved registers and a stack are used to manage parameters and return values

Arguments (in order)

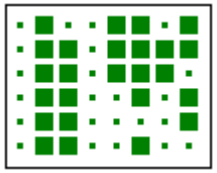
Integers

`%rdi, %rsi, %rdx, %rcx, %r8, %r9`

Floats – not used with B-Minor Compiler

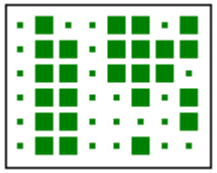
`%xmm0` through `%xmm7`

Imperative Lang Concept: Special Chars



- “Special characters” (syntatic sugar)
  - Symbols used to demarcate statements, functions, or other concepts in the imperative language
    - Python uses parentheses to identify functions and white-space to denote code blocks
    - C uses parentheses to identify functions and curly braces to denote code blocks
  - These symbols have no direct representation in the assembly code
    - For example the end of a block would be translated to a jump instruction

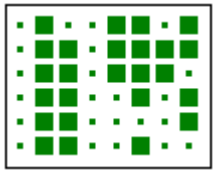
Imperative Lang Concept: Other Info



- Operational conventions
  - Built-in “rules of the road” that define how special situations are handled
    - E.g. Python starts running code outside of a function, C starts running code in main()
- Libraries/packages
  - Ready-to-use functions/methods that supply commonly needed functionality
    - E.g. access a file, get input from keyboard
- Interpretation or Compilation
  - The code we write has to be converted in some fashion so that the computer can execute it
  - Python uses interpretation, C uses compilation

Summary

Summary: Hardware → Software



- The von Neumann architecture led language designers to focus on interactions among the components (CPU, memory, input, output)
- Imperative languages were created to match the von Neumann architecture (paradigms align)
  - Use variables to access memory
  - Provide instructions to define execution order
- Assignment represents moving information between CPU, memory, input, output
- Iteration used for repetition (instructions nearby, memory cell for counter)
  - Discourages use of recursion (why?)

Homework

- Friday (9/18):
  - Compilers book Chapter 3
  - LLM: AI Coding Assistants…
- Sunday (9/20):
  - Journal entry for week 2
    - **2-1 Rule** (minimum 2 paragraphs, 1 quiz question)

